Machine learning

Name: Dana Hassan Bafaqih

ID: 445013918

Section number: 2

Table of Contents:

1. Breast Cancer Wisconsin
   - 1.1 Data Preprocessing
     - 1.1.1 Handling Missing Value
     - 1.1.2 Data Normalization
     - 1.1.3 Split the Dataset
   - 1.2 Model Building and Prediction
     - 1.2.1 Logistic Regression
     - 1.2.2 Naïve Bayes
   - 1.3 Model Evaluation
   - 1.4 Comparative Analysis
2. Adult Income Dataset
   - 2.1 Data Preprocessing
     - 2.1.1 Handling Missing Value
     - 2.1.2 Data Encoding
     - 2.1.3 Data Normalization
     - 2.1.4 Split the Dataset
   - 2.2 Model Building and Prediction
     - 2.2.1 Logistic Regression
     - 2.2.2 Naïve Bayes
   - 2.3 Model Evaluation
   - 2.4 Comparative Analysis

Import libraries:

```python
# For the dataset
import pandas as pd
import numpy as np

# For plotting
import matplotlib.pyplot as plt
import seaborn as sns

# For preprocessing
from sklearn.preprocessing import MinMaxScaler, LabelEncoder

# For Modelling
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB, BernoulliNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, recall_score, precision_score, confusion_matrix, classification_report, auc, roc_curve, roc_auc_score

# For Visualisation
import matplotlib.pyplot as plt
import seaborn as sns
```

1. Breast Cancer Wisconsin

About the dataset:

- The data contain 569 instance and 30 feature
- All the features are continuous
- Binary classification
- No missing value

Load the data and the first 5 rows

[44]

```python
# Read the data
Cancer  = pd.read_csv('/content/wdbc[1].data',
                      header = None,      # The data without header
                      delimiter=' *, *',
                      engine='python')
```

[21]

```python
# Add header to the dataset
Cancer.columns = ['ID','Diagnosis' , 'radius1', 'texture1', 'perimeter1', 'area1',
                  'smoothness1', 'compactness1', 'concavity1',
                  'concave_points1', 'symmetry1', 'fractal_dimension1',
                 'radius2','texture2', 'perimeter2',
                'area2', 'smoothness2', 'compactness2', 'concavity2',
                'concave_points2', 'symmetry2', 'fractal_dimension2', 'radius3',
                'texture3', 'perimeter3', 'area3', 'smoothness3',
                'compactness3', 'concavity3', 'concave_points3','symmetry3',
                'fractal_dimension3']
```

[22]

```python
# Print the first 5 rows
Cancer.head()
```

	ID	Diagnosis	radius1	texture1	perimeter1	area1	smoothness1	compactness1	concavity1	concave_points1	...	radius3	texture3	perimeter3	area3	smoothness3	compactness3	concavity3	concave_points3
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	25.38	17.33	184.60	2019.0	0.1622	0.6656	0.7119	0.2654
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	24.99	23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	23.57	25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	14.91	26.50	98.87	567.7	0.2098	0.8663	0.6869	0.2575
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	22.54	16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625

5 rows × 32 columns

1.1 Data Preprocessing

1.1.1 Handling Missing Value

First, I display information about the data using *.info*. It shows the columns, their data types, and it count the non-null values.

Then, I use *.isnull().sum()* to check for any missing value and return the sum of it.

From the *Non-Null count*, we can see that all columns has the same number of entries as the dataset instance, and from *isnull* we can see that there are no missing value.

```
# Show information about the data (columns, type, null count)
Cancer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   ID                 569 non-null    int64
 1   Diagnosis          569 non-null    object
 2   radius1            569 non-null    float64
 3   texture1           569 non-null    float64
 4   perimeter1         569 non-null    float64
 5   area1              569 non-null    float64
 6   smoothness1        569 non-null    float64
 7   compactness1       569 non-null    float64
 8   concavity1         569 non-null    float64
 9   concave_points1    569 non-null    float64
 10  symmetry1          569 non-null    float64
 11  fractal_dimension1 569 non-null    float64
 12  radius2            569 non-null    float64
 13  texture2           569 non-null    float64
 14  perimeter2         569 non-null    float64
 15  area2              569 non-null    float64
 16  smoothness2        569 non-null    float64
 17  compactness2       569 non-null    float64
 18  concavity2         569 non-null    float64
 19  concave_points2    569 non-null    float64
 20  symmetry2          569 non-null    float64
 21  fractal_dimension2 569 non-null    float64
 22  radius3            569 non-null    float64
 23  texture3           569 non-null    float64
 24  perimeter3         569 non-null    float64
 25  area3              569 non-null    float64
 26  smoothness3        569 non-null    float64
 27  compactness3       569 non-null    float64
 28  concavity3         569 non-null    float64
 29  concave_points3    569 non-null    float64
 30  symmetry3          569 non-null    float64
 31  fractal_dimension3 569 non-null    float64
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
```

```
# Check for missing vlues value
Cancer.isnull().sum()
```

	0
ID	0
Diagnosis	0
radius1	0
texture1	0
perimeter1	0
area1	0
smoothness1	0
compactness1	0
concavity1	0
concave_points1	0
symmetry1	0
fractal_dimension1	0
radius2	0
texture2	0
perimeter2	0
area2	0
smoothness2	0
compactness2	0
concavity2	0
concave_points2	0
symmetry2	0
fractal_dimension2	0
radius3	0
texture3	0
perimeter3	0
area3	0
smoothness3	0
compactness3	0
concavity3	0
concave_points3	0
symmetry3	0
fractal_dimension3	0

dtype: int64

1.1.2 Data Normalization

Before applying normalization, I check if the data needs it using *.describe()* to see the minimum and maximum values.

I observed that some columns have large differences between their min and max values, so I apply it.

```
# Show statistical summary
Cancer.describe()
```

	0	2	3	4	5	6	7	8	9	10	...	22	23	24	25	26	27	56
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	...	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162	...	16.269190	25.677223	107.261213	880.583128	0.132369	0.254265	
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	...	4.833242	6.146258	33.602542	569.356993	0.022832	0.157336	
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	...	7.930000	12.020000	50.410000	185.200000	0.071170	0.027290	
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	...	13.010000	21.080000	84.110000	515.300000	0.116600	0.147200	
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	...	14.970000	25.410000	97.660000	686.500000	0.131300	0.211900	
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	...	18.790000	29.720000	125.400000	1084.000000	0.146000	0.339100	
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	...	36.040000	49.540000	251.200000	4254.000000	0.222600	1.058000	

8 rows × 31 columns

Because normalization only applies to numerical features, and all our features are numeric, there is no issue here. We just need to split the dataset into the Feature(X) and Target(y)

```
# Split features and target
X = Cancer.drop(['Diagnosis'], axis=1)  # Feature
y = Cancer['Diagnosis']                 # Target
```

```
# Applying normalization to scale features between 0 and 1.
scaler = MinMaxScaler()
normX = scaler.fit_transform(X)
```

1.1.3 Split the Dataset

To train the model and then test its performance, we need to spilt the data into Training and Testing sets for both x and y.

I split into 30% for testing and 70% for training.

```
#Split the dataset into training and testing sets (70% train, 30% test).
X_train, X_test, y_train, y_test = train_test_split(
    normX, y, test_size=0.3, random_state=42
    )
```

1.2 Model Building and Prediction

1.2.1 Logistic Regression Model

It's a binary classification algorithm that predicts probabilities and handle any type of features as long as the target variable is binary.

```python
# Build a Logistic Regression Classifier
model1 = LogisticRegression()

# Model training
model1.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression()
```

1.2.1.1 Logistic Regression prediction

```python
# Predict the classes for the test set
y_pred_LOG = model1.predict(X_test)
print("Predict classes",y_pred_LOG)

# Predict the probabilities for the test set
y_probs_LOG = model1.predict_proba(X_test)[:, 1]
print("Predict probabilities",y_probs_LOG)
```

```
Predict classes ['M' 'M' 'M' 'B' 'B' 'M' 'M' 'M' 'B' 'B' 'B' 'M' 'B' 'B' 'B' 'B' 'B' 'B'
 'B' 'M' 'M' 'B' 'M' 'B' 'B' 'B' 'B' 'B' 'B' 'M' 'B' 'B' 'B' 'M' 'B' 'B'
 'M' 'B' 'M' 'B' 'B' 'M' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'M' 'M' 'B' 'B'
 'B' 'B' 'B' 'M' 'M' 'B' 'B' 'M' 'M' 'B' 'B' 'B' 'M' 'M' 'B' 'B' 'M' 'M'
 'B' 'B' 'B' 'B' 'B' 'M' 'B' 'B' 'M' 'B' 'B' 'M' 'M' 'M' 'M' 'M' 'B' 'B'
 'B' 'B' 'B' 'B' 'B' 'B' 'M' 'M' 'B' 'M' 'M' 'B' 'M' 'M' 'M' 'B' 'B' 'M'
 'B' 'B' 'M' 'B' 'B' 'M' 'B' 'M' 'B' 'B' 'B' 'M' 'B' 'B' 'B' 'M' 'B' 'M'
 'M' 'B' 'B' 'M' 'M' 'B' 'M' 'B' 'M' 'M' 'B' 'B' 'B' 'M' 'B' 'M' 'B' 'B'
 'M' 'B' 'M' 'M' 'M' 'B' 'M' 'B' 'B' 'B' 'B' 'M' 'B' 'B' 'B' 'B' 'B' 'B'
 'M' 'B' 'M' 'B' 'B' 'B' 'B' 'M' 'B']
Predict probabilities [0.76125391 0.99999806 0.99654259 0.05298831 0.1026459  0.99914723
 0.99999351 0.77226107 0.1263391  0.17949404 0.1274113  0.98696369
 0.065795   0.17955394 0.17374983 0.07102232 0.05572752 0.09859031
 0.27161265 0.99768206 0.80406662 0.28457894 0.99996435 0.2024631
 0.09401836 0.04465897 0.09571956 0.08293359 0.12168117 0.99999931
 0.1120525  0.16526632 0.38606675 0.65432786 0.17385465 0.23762
 0.98043753 0.31556666 0.99982488 0.24049491 0.17917517 0.97880776
 0.12549892 0.33431813 0.06835205 0.38023561 0.26758596 0.20017746
 0.40450286 0.1046592  0.99947891 0.99988182 0.16053123 0.05604183
 0.14556451 0.10005362 0.19534541 0.99980607 0.53946568 0.153297
```

1.2.2 Naïve Bayes Model

It's a binary classification algorithm that uses the principles of probability to perform classification tasks. It assumes that features are independent.

I chose Gaussian Naïve Bayes because all the features are continuous, and model works well with continuous data.

```
# Build a Gaussian NB Classifier
model2 = GaussianNB()

# Model training
model2.fit(X_train, y_train)
```

```
GaussianNB
GaussianNB()
```

1.2.2.1 Naïve Bayes Prediction

```
# Predict the classes for the test set
y_pred_GNB = model2.predict(X_test)
print("Predict classes",y_pred_GNB)

# Predict the probabilities for the test set
y_probs_GNB = model2.predict_proba(X_test)[:, 1]
print("Predict probabilities",y_probs_GNB)
```

```
Predict classes ['B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B'
 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B'
 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B'
 'B' 'M' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B'
 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B'
 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B'
 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'M'
 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B'
 'B' 'B' 'B' 'M' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B'
 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B']
Predict probabilities [0.35864488 0.37797488 0.3663255  0.35668516 0.35553285 0.37844615
 0.38586992 0.3687856  0.35884676 0.36003835 0.35922864 0.36931402
 0.36207819 0.36479431 0.36646704 0.37159942 0.36112524 0.3535761
 0.35046653 0.37859736 0.36104595 0.35704438 0.38274214 0.35387799
 0.3558532  0.35902928 0.35849267 0.35376663 0.35829053 0.37416113
 0.35510614 0.35673459 0.35382931 0.36976849 0.3545441  0.36459621
 0.36667137 0.35725072 0.37404763 0.36151845 0.35440261 0.36554585
 0.35577822 0.35738182 0.35659743 0.35618846 0.35294885 0.35309994
 0.36593183 0.3586221  0.37112316 0.37523999 0.3592232  0.35924404
 0.35635994 0.65495222 0.3541722  0.39313131 0.36000548 0.35605121
 0.35883284 0.38503284 0.38266139 0.36122786 0.35698724 0.36693007
```

1.3 Model Evaluation

1.3.1 Logistic regression

I calculate the evaluation metrics for the model to check its performance.

```
print("\nLogistic Regression")

# Model Evaluation Metrics
print("\nAccuracy:", accuracy_score(y_test, y_pred_LOG))
print("\nPrecision:", precision_score(y_test, y_pred_LOG,pos_label="M"))
print("\nRecall:", recall_score(y_test, y_pred_LOG, pos_label="M"))
print("\nF1 Score:", f1_score(y_test, y_pred_LOG, pos_label="M"))
print("\nAUC - ROC Score:", roc_auc_score(y_test, y_probs_LOG))

print("\n-------------------------------------------------------")

# Classification report
print("\nLogistic Regression Classification Report:")
print(classification_report(y_test, y_pred_LOG))
```

```
Logistic Regression

Accuracy: 0.9766081871345029

Precision: 1.0

Recall: 0.9365079365079365

F1 Score: 0.9672131147540983

AUC - ROC Score: 0.9982363315696648

-------------------------------------------------------

Logistic Regression Classification Report:
              precision    recall  f1-score   support

           B       0.96      1.00      0.98       108
           M       1.00      0.94      0.97        63

    accuracy                           0.98       171
   macro avg       0.98      0.97      0.97       171
weighted avg       0.98      0.98      0.98       171
```

As we can see, the model achieved:

Accuracy = 97% The model predicts correctly in most cases

Precision = 100% Every time the model predicts positive it's actually positive (no FP).

Recall = 93% The model correctly classifies most of the actual positive cases (few FN)

F1 score = 96% Balance performance between Precision and Recall

AUC-ROC = 99% The model has perfect distinguish between positive and negative cases

1.3.2 Naïve Bayes

```python
print("\nGaussian Naive Bayes")

# Model Evaluation Metrics
print("\nAccuracy:", accuracy_score(y_test, y_pred_GNB))
print("\nPrecision:", precision_score(y_test, y_pred_GNB, pos_label="M"))
print("\nRecall:", recall_score(y_test, y_pred_GNB, pos_label="M"))
print("\nF1 Score:", f1_score(y_test, y_pred_GNB, pos_label="M"))
print("\nAUC - ROC Score:", roc_auc_score(y_test, y_probs_GNB))

print("\n----------------------------------------------------")

# Classification report
print("\nGaussian Naive Bayes Classification Report:")
print(classification_report(y_test, y_pred_GNB))
```

```
Gaussian Naive Bayes

Accuracy: 0.935672514619883

Precision: 0.9193548387096774

Recall: 0.9047619047619048

F1 Score: 0.912

AUC - ROC Score: 0.9926513815402704

----------------------------------------------------

Gaussian Naive Bayes Classification Report:
              precision    recall  f1-score   support

           B       0.94      0.95      0.95       108
           M       0.92      0.90      0.91        63

    accuracy                           0.94       171
   macro avg       0.93      0.93      0.93       171
weighted avg       0.94      0.94      0.94       171
```

As we can see, the model achieved:

Accuracy = 93% The model predicts correctly in most cases

Precision = 91% Most of the predicted positive are actually positive (few FP).

Recall = 90% The model correctly classifies most of the actual positive cases (few FN)

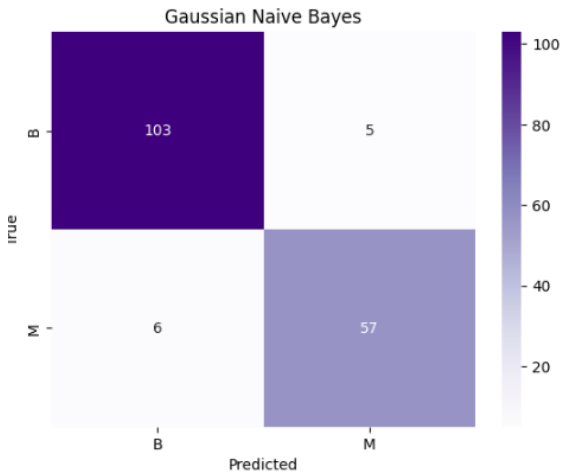
F1 score = 91% Balance performance between Precision and Recall

AUC-ROC = 99% The model has perfect distinguish between positive and negative cases

1.3.3 Confusion Matrix

Naïve Bayes

```
# Visualize the confusion_matrix for Gaussian Naive Bayes
cm = confusion_matrix(y_test, y_pred_GNB, labels=['B','M'])
plt.figure()
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples', xticklabels=['B', 'M'],yticklabels=['B', 'M'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Gaussian Naive Bayes')
plt.show()
```

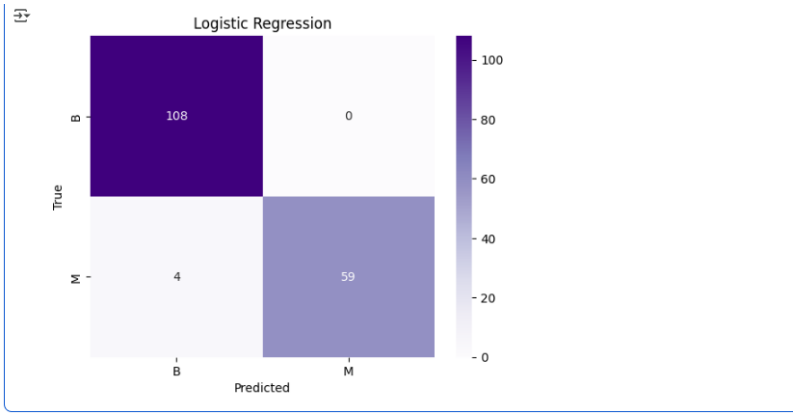


The result:

TP: 57 FP: 5 TN: 103 FN: 6

Logistic regression

```
# Visualize the confusion_matrix for Logistic Regression
cm = confusion_matrix(y_test, y_pred_LOG, labels=['B','M'])
plt.figure()
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples',xticklabels=['B', 'M'],yticklabels=['B', 'M'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Logistic Regression')
plt.show()
```



The result:

TP: 59 FP: 0 TN: 108 FN: 4

1.3.4 ROC Curve

It's a graphical representation of the performance of both logistic regression and Naïve Bayes across various threshold values.

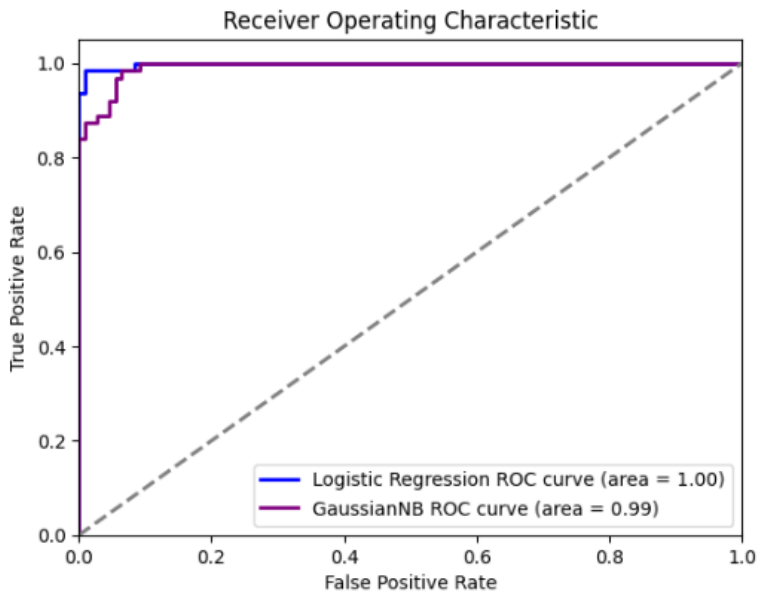
The curve represents the trade off between sensitivity (TPR) and specificity (FPR).

```python
# Plotting the ROC Curve
fpr_LOG, tpr_LOG, thresholds = roc_curve(y_test, y_probs_LOG, pos_label="M")
roc_auc_LOG = auc(fpr_LOG, tpr_LOG)

fpr_GNB, tpr_GNB, thresholds = roc_curve(y_test, y_probs_GNB, pos_label="M")
roc_auc_GNB = auc(fpr_GNB, tpr_GNB)


plt.figure()
plt.plot(fpr_LOG, tpr_LOG, color='blue', lw=2, label=f'Logistic Regression ROC curve (area = {roc_auc_LOG:.2f})')
plt.plot(fpr_GNB, tpr_GNB, color='purple', lw=2, label=f'GaussianNB ROC curve (area = {roc_auc_GNB:.2f})')

plt.plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()
```



The logistic regression achieved a score of 1.0, which means the model has perfect ability to discriminate between positive and negative classes

On the other hand, the Gaussian naïve bayes achieved a score of 0.99, which also indicate good performance.

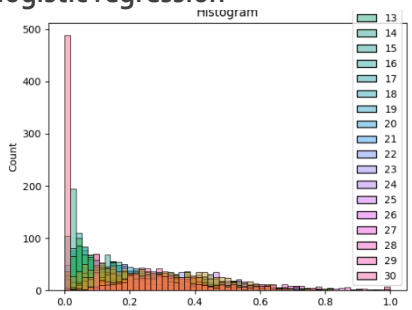
1.4 Comparative Analysis

● Compare the performance of Logistic Regression vs. Naive Bayes on each dataset.

The logistic regression model achieved higher accuracy of 0.97, while the Gaussian naïve bayes achieved 0.93, which is still good but lower than logistic regression.
In the precision, logistic regression showed a perfect score 1.0, indicating that the model always predict the positive class with no mistake(FP), while Gaussian achieved 0.9, meaning it predict some of the positive wrong.
Overall, the logistic regression demonstrated better performance and higher score across all the evaluation metrics

● Discuss how dataset characteristics (feature types, distributions) influenced results.

All the features are continuous, which means Gaussian naïve bayes is the best for the data from the naïve bayes family, but it assume the features follow Gaussian distribution (normal), and they actually positive skew. This could be the reason why the model achieved lower scores than the logistic regression



● Explore how preprocessing choices affected model performance.

I applied normalization on the data because some features had large differences between their values.
When I tested the model without normalization, the logistic regression achieved accuracy that a lower score 0.92 and a precision score of 0.90, while Gaussian naïve bayes achieved an accuracy of 0.63, which not good.
So, normalization had a good effect on the models.

```
Logistic Regression

Accuracy: 0.9298245614035088

Precision: 0.9047619047619048

Recall: 0.9047619047619048

F1 Score: 0.9047619047619048

AUC - ROC Score: 0.9584068195179306

-----------------------------------------------------

Logistic Regression Classification Report:
              precision    recall  f1-score   support

           B       0.94      0.94      0.94       108
           M       0.90      0.90      0.90        63

    accuracy                           0.93       171
   macro avg       0.92      0.92      0.92       171
weighted avg       0.93      0.93      0.93       171
```

```
Gaussian Naive Bayes

Accuracy: 0.6374269005847953

Precision: 0.6666666666666666

Recall: 0.031746031746031744

F1 Score: 0.06060606060606061

AUC - ROC Score: 0.9409171075837742

-----------------------------------------------------

Gaussian Naive Bayes Classification Report:
              precision    recall  f1-score   support

           B       0.64      0.99      0.78       108
           M       0.67      0.03      0.06        63

    accuracy                           0.64       171
   macro avg       0.65      0.51      0.42       171
weighted avg       0.65      0.64      0.51       171
```

● Reflect on which model you would recommend for each dataset and why.

I recommend logistic regression model because it gives higher performance and fits with the data well, while the GaussianNB assumes a normal distribution which not satisfied in this data.

2. Adult Income Dataset

About the dataset:

- The data contain 48842 instance and 14 feature
- The feature mix of integer and categorical
- Binary classification
- There are missing value

```python
# Read the data
adult = pd.read_csv('/content/adult[1].data',
                    header = None,              # The data without header
                    delimiter=' *, *', engine='python')
```

```python
# Add header to the dataset
adult.columns = ['age', 'workclass', 'fnlwgt', 'education', 'education_num',
                 'marital_status', 'occupation', 'relationship',
                 'race', 'sex', 'capital_gain', 'capital_loss',
                 'hours_per_week', 'native_country', 'income']
```

```python
# Print the first 5 rows
adult.head()
```

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_week	native_country	income
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K

2.1 Handling Missing Value

Display information about the data and check missing value

```python
# Show information about the data (columns, type, null count)
adult.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32561 entries, 0 to 32560
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             32561 non-null  int64
 1   workclass       32561 non-null  object
 2   fnlwgt          32561 non-null  int64
 3   education       32561 non-null  object
 4   education_num   32561 non-null  int64
 5   marital_status  32561 non-null  object
 6   occupation      32561 non-null  object
 7   relationship    32561 non-null  object
 8   race            32561 non-null  object
 9   sex             32561 non-null  object
 10  capital_gain    32561 non-null  int64
 11  capital_loss    32561 non-null  int64
 12  hours_per_week  32561 non-null  int64
 13  native_country  32561 non-null  object
 14  income          32561 non-null  object
dtypes: int64(6), object(9)
memory usage: 3.7+ MB
```

```python
# Check for missing vlues value
adult.isnull().sum()
```

	0
age	0
workclass	0
fnlwgt	0
education	0
education_num	0
marital_status	0
occupation	0
relationship	0
race	0
sex	0
capital_gain	0
capital_loss	0
hours_per_week	0
native_country	0
income	0

dtype: int64

Even though it shows no missing value (null), but the data source indicates that some values are missing.

This means they may not be null, but the column contains invalid or missing values.

So, I check the unique values for columns that may contain missing or invalid values.

```python
# Check the unique values for columns that may contain missing or invalid values
for col in ['workclass', 'occupation', 'native_country']:
    print(f"Column: {col}")
    print(adult[col].unique())
    print("\n")
```

```
Column: workclass
['State-gov' 'Self-emp-not-inc' 'Private' 'Federal-gov' 'Local-gov' '?'
 'Self-emp-inc' 'Without-pay' 'Never-worked']


Column: occupation
['Adm-clerical' 'Exec-managerial' 'Handlers-cleaners' 'Prof-specialty'
 'Other-service' 'Sales' 'Craft-repair' 'Transport-moving'
 'Farming-fishing' 'Machine-op-inspct' 'Tech-support' '?'
 'Protective-serv' 'Armed-Forces' 'Priv-house-serv']


Column: native_country
['United-States' 'Cuba' 'Jamaica' 'India' '?' 'Mexico' 'South'
 'Puerto-Rico' 'Honduras' 'England' 'Canada' 'Germany' 'Iran'
 'Philippines' 'Italy' 'Poland' 'Columbia' 'Cambodia' 'Thailand' 'Ecuador'
 'Laos' 'Taiwan' 'Haiti' 'Portugal' 'Dominican-Republic' 'El-Salvador'
 'France' 'Guatemala' 'China' 'Japan' 'Yugoslavia' 'Peru'
 'Outlying-US(Guam-USVI-etc)' 'Scotland' 'Trinadad&Tobago' 'Greece'
 'Nicaragua' 'Vietnam' 'Hong' 'Ireland' 'Hungary' 'Holand-Netherlands']
```

It shows that some entries contain the '?' value so we need to handle them.
I replaced the '?' values with the mode to fill the columns

```python
# Replace the '?' with the mode because these are categorical features
for col in ['workclass', 'occupation', 'native_country']:
  adult[col] = adult[col].replace('?', adult[col].mode()[0])
```

I checked the unique values to ensure that there are no '?' entries

```python
# Check again
for col in ['workclass', 'occupation', 'native_country']:
    print(f"Column: {col}")
    print(adult[col].unique())
    print("\n")
```

```
Column: workclass
['State-gov' 'Self-emp-not-inc' 'Private' 'Federal-gov' 'Local-gov'
 'Self-emp-inc' 'Without-pay' 'Never-worked']


Column: occupation
['Adm-clerical' 'Exec-managerial' 'Handlers-cleaners' 'Prof-specialty'
 'Other-service' 'Sales' 'Craft-repair' 'Transport-moving'
 'Farming-fishing' 'Machine-op-inspct' 'Tech-support' 'Protective-serv'
 'Armed-Forces' 'Priv-house-serv']


Column: native_country
['United-States' 'Cuba' 'Jamaica' 'India' 'Mexico' 'South' 'Puerto-Rico'
 'Honduras' 'England' 'Canada' 'Germany' 'Iran' 'Philippines' 'Italy'
 'Poland' 'Columbia' 'Cambodia' 'Thailand' 'Ecuador' 'Laos' 'Taiwan'
 'Haiti' 'Portugal' 'Dominican-Republic' 'El-Salvador' 'France'
 'Guatemala' 'China' 'Japan' 'Yugoslavia' 'Peru'
 'Outlying-US(Guam-USVI-etc)' 'Scotland' 'Trinadad&Tobago' 'Greece'
 'Nicaragua' 'Vietnam' 'Hong' 'Ireland' 'Hungary' 'Holand-Netherlands']
```

There are no '?' values remaining

2.1.1 Data Encoding

Because there are categorical features, we need to encode them so that the models can understand them.

The 'sex' column is binary, so I used LabelEncoder to convert the values to binary (0,1) in the same columns

Other features have no order in the values, so I converted them using One-Hot Encoding, which creates a column for each category, assigning 1 if the row has that category and 0 if not.

```python
# Encode the 'sex' column with LabelEncoder because it's' binary
adult['sex'] = LabelEncoder().fit_transform(adult['sex'])

# Encode remaining categorical features with One-Hot because there is no ordering in the data
adultE = pd.get_dummies( adult, columns=['workclass', 'education', 'marital_status', 'occupation',
             'relationship', 'race', 'native_country'], dtype=int)

print(adultE.head())
```

```
   age  fnlwgt  education_num  sex  capital_gain  capital_loss  \
0   39   77516             13    1          2174             0
1   50   83311             13    1             0             0
2   38  215646              9    1             0             0
3   53  234721              7    1             0             0
4   28  338409             13    0             0             0

   hours_per_week income  workclass_Federal-gov  workclass_Local-gov  ...  \
0              40  <=50K                      0                    0  ...
1              13  <=50K                      0                    0  ...
2              40  <=50K                      0                    0  ...
3              40  <=50K                      0                    0  ...
4              40  <=50K                      0                    0  ...

   native_country_Portugal  native_country_Puerto-Rico  \
0                        0                           0
1                        0                           0
2                        0                           0
3                        0                           0
4                        0                           0

   native_country_Scotland  native_country_South  native_country_Taiwan  \
0                        0                     0                      0
1                        0                     0                      0
```

2.1.2 Data Normalization

I applied the same steps as I did with the previous dataset.

I observed that some columns have large differences their min and max values, so I apply it.

```
# Show statistical summary
adultE.describe()
```

	age	fnlwgt	education_num	sex	capital_gain	capital_loss	hours_per_week	workclass_Federal-gov	workclass_Local-gov	workclass_Never-worked	...	native_country_Portugal	native_cou
count	32561.000000	3.256100e+04	32561.000000	32561.000000	32561.000000	32561.000000	32561.000000	32561.000000	32561.000000	32561.000000	...	32561.000000	
mean	38.581647	1.897784e+05	10.080679	0.669205	1077.648844	87.303830	40.437456	0.029483	0.064279	0.000215	...	0.001136	
std	13.640433	1.055500e+05	2.572720	0.470506	7385.292085	402.960219	12.347429	0.169159	0.245254	0.014661	...	0.033691	
min	17.000000	1.228500e+04	1.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	...	0.000000	
25%	28.000000	1.178270e+05	9.000000	0.000000	0.000000	0.000000	40.000000	0.000000	0.000000	0.000000	...	0.000000	
50%	37.000000	1.783560e+05	10.000000	1.000000	0.000000	0.000000	40.000000	0.000000	0.000000	0.000000	...	0.000000	
75%	48.000000	2.370510e+05	12.000000	1.000000	0.000000	0.000000	45.000000	0.000000	0.000000	0.000000	...	0.000000	
max	90.000000	1.484705e+06	16.000000	1.000000	99999.000000	4356.000000	99.000000	1.000000	1.000000	1.000000	...	1.000000	

8 rows × 104 columns

```
# Split features and target
X = adultE.drop(['income'], axis=1)
y = adultE['income']
```

```
# Applying normalization to scale features between 0 and 1.
scaler = MinMaxScaler()
normX = scaler.fit_transform(X)
```

1.2.3 Split the Dataset

```
#Split the dataset into training and testing sets (70% train, 30% test).
X_train, X_test, y_train, y_test = train_test_split(
    normX, y, test_size=0.3, random_state=42
    )
```

2.2 Model Building and Prediction

2.2.1 Logistic Regression

```
# Build a Logistic Regression Classifier
model1 = LogisticRegression()

# Model training
model1.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression()
```

```
# Predict the classes for the test set
y_pred_LOG = model1.predict(X_test)
print("Predict classes",y_pred_LOG)

# Predict the probabilities for the test set
y_probs_LOG = model1.predict_proba(X_test)[:, 1]
print("Predict probabilities",y_probs_LOG)
```

```
Predict classes ['<=50K' '<=50K' '>50K' ... '<=50K' '<=50K' '>50K']
Predict probabilities [0.02733206 0.48853076 0.69515524 ... 0.01907129 0.4303986  0.84006171]
```

2.2.2 Naïve Bayes

Gaussian naïve bayes

```
# Build a Gaussian NB Classifier
model2 = GaussianNB()

# Model training
model2.fit(X_train, y_train)
```

```
GaussianNB
GaussianNB()
```

```
# Predict the classes for the test set
y_pred_GNB = model2.predict(X_test)
print("Predict classes",y_pred_GNB)

# Predict the probabilities for the test set
y_probs_GNB = model2.predict_proba(X_test)[:, 1]
print("Predict probabilities",y_probs_GNB)
```

```
Predict classes ['>50K' '>50K' '>50K' ... '<=50K' '>50K' '>50K']
Predict probabilities [1.         1.         1.         ... 0.18691957 1.         1.        ]
```

Bernoulli naïve bayes

```python
# Build a Bernoulli NB Classifier
model3 = BernoulliNB()

# Model training
model3.fit(X_train, y_train)
```

```
BernoulliNB
BernoulliNB()
```

```python
# Predict the classes for the test set
y_pred_BNB = model3.predict(X_test)
print("Predict classes",y_pred_BNB)

# Predict the probabilities for the test set
y_probs_BNB = model3.predict_proba(X_test)[:, 1]
print("Predict probabilities",y_probs_BNB)
```

```
Predict classes ['<=50K' '>50K' '>50K' ... '<=50K' '>50K' '>50K']
Predict probabilities [5.88335643e-04 7.69442184e-01 9.40227482e-01 ... 4.04613710e-04
 8.91001389e-01 9.92575028e-01]
```

2.3 Model Evaluation

2.3.1 Logistic Regression

```
print("\nLogistic Regression")

# Model Evaluation Metrics
print("\nAccuracy:", accuracy_score(y_test, y_pred_LOG))
print("\nPrecision:", precision_score(y_test, y_pred_LOG, pos_label='>50K'))
print("\nRecall:", recall_score(y_test, y_pred_LOG, pos_label='>50K'))
print("\nF1 Score:", f1_score(y_test, y_pred_LOG, pos_label='>50K'))
print("\nAUC - ROC Score:", roc_auc_score(y_test, y_probs_LOG))

print("-----------------------------------------------")

# Classification report
print("\nLogistic Regression Classification Report:")
print(classification_report(y_test, y_pred_LOG))
```

```
Logistic Regression

Accuracy: 0.8506500153546934

Precision: 0.7272727272727273

Recall: 0.5911840968020743

F1 Score: 0.6522050059594756

AUC - ROC Score: 0.902076185143126
-----------------------------------------------

Logistic Regression Classification Report:
              precision    recall  f1-score   support

       <=50K       0.88      0.93      0.90      7455
        >50K       0.73      0.59      0.65      2314

    accuracy                           0.85      9769
   macro avg       0.80      0.76      0.78      9769
weighted avg       0.84      0.85      0.85      9769
```

As we can see, the model achieved:

Accuracy = 85% The model predicts correctly in most cases

Precision = 72% When the model predicts positive, it's actually positive, but there are some FP

Recall = 59% The model does not correctly classifies all the actual positive cases, there are several FN

F1 score = 65% The balance between Precision and Recall not very good

AUC-ROC = 90% The model has perfect distinguish between positive and negative cases

2.3.2 Naïve Bayes

Because the features are binary, and continues values, I tried both Gaussian and Bernoulli naïve bayes to see which achieves a better score.
As we know, Gaussian naïve bayes assume that the features follow a normal distribution and works when the features are continuous
On the other hand, Bernoulli naïve bayes works when features are binary

Gaussian Naïve Bayes

```python
print("Gaussian Naive Bayes\n")

# Model Evaluation Metrics
print("\nAccuracy:", accuracy_score(y_test, y_pred_GNB))
print("\nPrecision:", precision_score(y_test, y_pred_GNB, pos_label='>50K'))
print("\nRecall:", recall_score(y_test, y_pred_GNB, pos_label='>50K'))
print("\nF1 Score:", f1_score(y_test, y_pred_GNB, pos_label='>50K'))
print("\nAUC - ROC Score:", roc_auc_score(y_test, y_probs_GNB))

print("----------------------------------------------")

# Classification report
print("\nGaussian Naive Bayes Classification Report:")
print(classification_report(y_test, y_pred_GNB))
```

```
Gaussian Naive Bayes


Accuracy: 0.5011771931620432

Precision: 0.317864768683274

Recall: 0.9649956784788245

F1 Score: 0.47820965842167257

AUC - ROC Score: 0.7131689010467298
----------------------------------------------

Gaussian Naive Bayes Classification Report:
              precision    recall  f1-score   support

       <=50K       0.97      0.36      0.52      7455
        >50K       0.32      0.96      0.48      2314

    accuracy                           0.50      9769
   macro avg       0.64      0.66      0.50      9769
weighted avg       0.82      0.50      0.51      9769
```

As we can see, the model achieved:

Accuracy = 50% The model predicts correctly in only half of the cases, which is low

Precision = 31% When the model predicts positive, it's not actually positive in most of the cases (large FP)

Recall = 94% The model correctly classifies most of the actual positive cases (few FN)

F1 score = 47% The balance between Precision and Recall not very good, because the Precision is low

AUC-ROC = 71% The model has perfect distinguish between positive and negative cases

Overall, the performance is not good.

Bernoulli naïve bayes

```python
print("Bernoulli Naive Bayes\n")

# Evaluation
print("\nAccuracy:", accuracy_score(y_test, y_pred_BNB))
print("\nPrecision:", precision_score(y_test, y_pred_BNB, pos_label='>50K'))
print("\nRecall:", recall_score(y_test, y_pred_BNB, pos_label='>50K'))
print("\nF1 Score:", f1_score(y_test, y_pred_BNB, pos_label='>50K'))

print("\nAUC - ROC Score:", roc_auc_score(y_test, y_probs_BNB))

print("-----------------------------------------------")

print("\nBernoulli Naive Bayes Classification Report:")
print(classification_report(y_test, y_pred_BNB))
```

```
Bernoulli Naive Bayes


Accuracy: 0.7615927935305559

Precision: 0.49796029371770467

Recall: 0.7912705272255834

F1 Score: 0.6112502086463028

AUC - ROC Score: 0.870074900570232
-----------------------------------------------

Bernoulli Naive Bayes Classification Report:
              precision    recall  f1-score   support

       <=50K       0.92      0.75      0.83      7455
        >50K       0.50      0.79      0.61      2314

    accuracy                           0.76      9769
   macro avg       0.71      0.77      0.72      9769
weighted avg       0.82      0.76      0.78      9769
```

As we can see, the model achieved:

Accuracy = 76% **The model predicts correctly in most cases.**

Precision = 49% **When the model predicts positive, it's actually positive in half of the time (large FP)**

Recall = 79% **The model correctly classifies most of the actual positive cases (few FN)**

F1 score = 61% **The balance between Precision and Recall not very good**

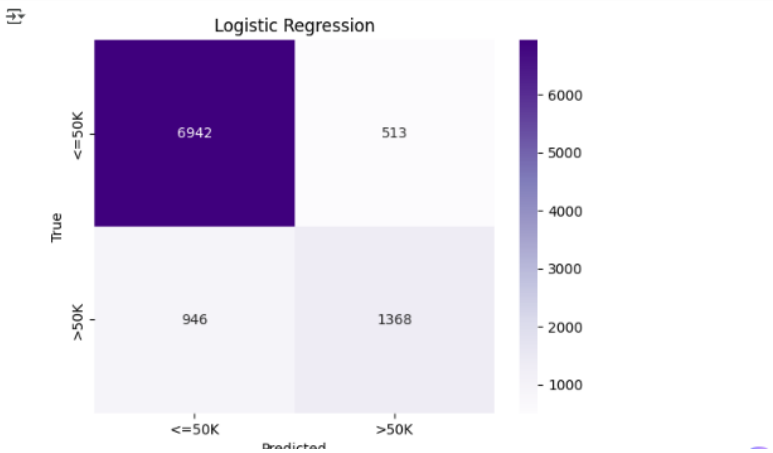
AUC-ROC = 87% **The model has good distinguish between positive and negative cases**

The model is butter that Gaussian.

2.3.3 Confusion Matrix

Logistic Regression

```
# Visualize the confusion matrix for Logistic Regression
cm = confusion_matrix(y_test, y_pred_LOG, labels = ['<=50K', '>50K'])
plt.figure()
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples', xticklabels=['<=50K', '>50K'], yticklabels=['<=50K', '>50K'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Logistic Regression')
plt.show()
```

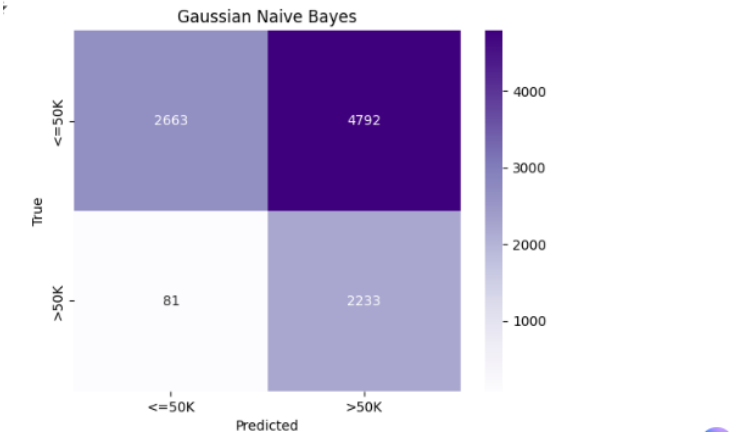


The result:

TP: 1368 FP: 513 TN: 6942 FN: 946

Naïve Bayes

```
# Visualize the confusion matrix for Gaussian Naive Bayes
cm = confusion_matrix(y_test, y_pred_GNB, labels = ['<=50K', '>50K'])
plt.figure()
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples', xticklabels=['<=50K', '>50K'], yticklabels=['<=50K', '>50K'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Gaussian Naive Bayes')
plt.show()
```

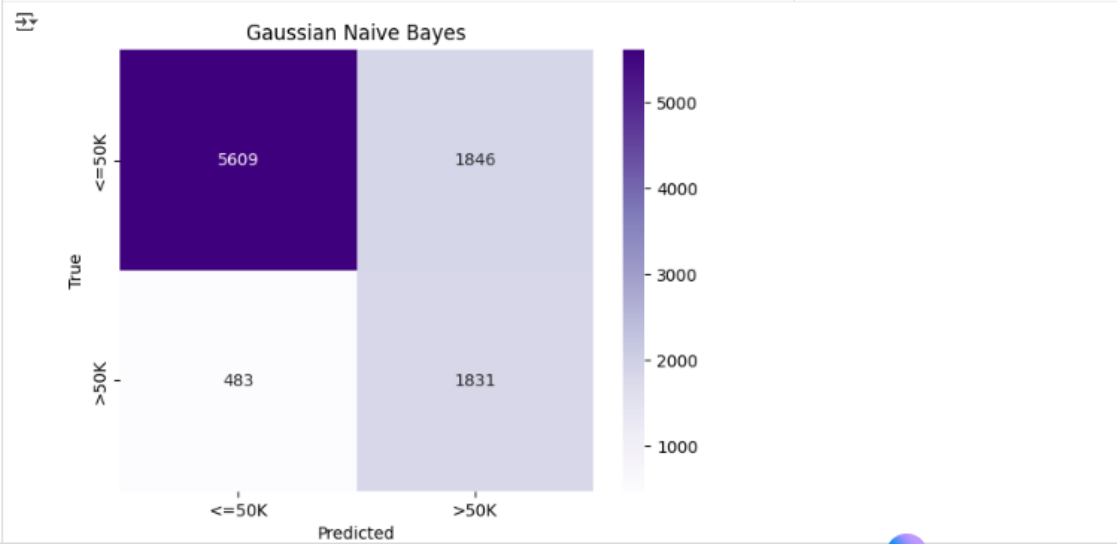


The result:

TP: 2233 FP: 4792 TN: 2663 FN: 81

Bernoulli naïve bayes

```
# Visualize the confusion matrix for Bernoulli  Naive Bayes
cm = confusion_matrix(y_test, y_pred_BNB, labels = ['<=50K', '>50K'])
plt.figure()
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples', xticklabels=['<=50K', '>50K'], yticklabels=['<=50K', '>50K'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Gaussian Naive Bayes')
plt.show()
```



The result:

TP: 1831 FP: 1846 TN: 5609 FN: 483

2.3.4 ROC Curve

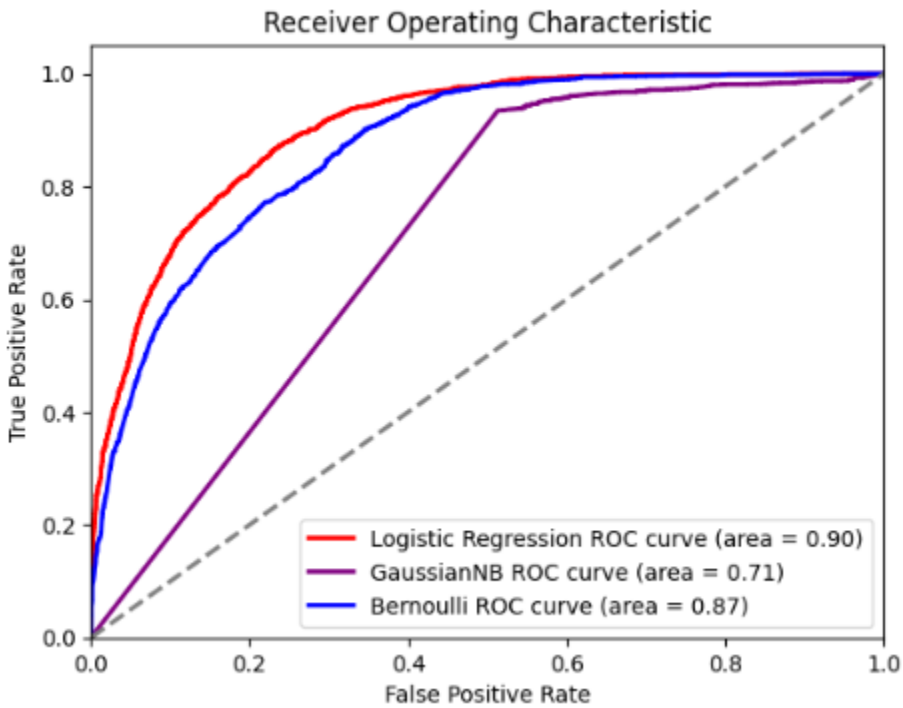
```
# Plotting the ROC Curve
fpr_LOG, tpr_LOG, thresholds = roc_curve(y_test, y_probs_LOG, pos_label=">50K")
roc_auc_LOG = auc(fpr_LOG, tpr_LOG)

fpr_GNB, tpr_GNB, thresholds = roc_curve(y_test, y_probs_GNB, pos_label=">50K")
roc_auc_GNB = auc(fpr_GNB, tpr_GNB)

fpr_BNB, tpr_BNB, thresholds = roc_curve(y_test, y_probs_BNB, pos_label=">50K")
roc_auc_BNB = auc(fpr_BNB, tpr_BNB)

plt.figure()
plt.plot(fpr_LOG, tpr_LOG, color='red', lw=2, label=f'Logistic Regression ROC curve (area = {roc_auc_LOG:.2f})')
plt.plot(fpr_GNB, tpr_GNB, color='purple', lw=2, label=f'GaussianNB ROC curve (area = {roc_auc_GNB:.2f})')
plt.plot(fpr_BNB, tpr_BNB, color='blue', lw=2, label=f'Bernoulli ROC curve (area = {roc_auc_BNB:.2f})')

plt.plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()
```



The logistic regression achieved a score of 0.90, which means the model has perfect ability to discriminate between positive and negative classes

The Gaussian naïve bayes achieved a score of 0.71, which indicate moderate performance.

The Bernoulli naïve bayes achieved a score of 0.87, which indicate good performance and better discrimination that the Gaussian.

2.4 Comparative Analysis

- **Compare the performance of Logistic Regression vs. Naive Bayes on each dataset.**

Here also the Logistic Regression achieved higher scores across all metrics.
This may be because the features are both numerical and categorical, and Logistic Regression can handle any type of features. On the other hand, both naïve bayes models that I used (Gaussian, Bernoulli) handle specific type of feature, Bernoulli works with binary and Gaussian work withs continues.

- **Discuss how dataset characteristics (feature types, distributions) influenced results.**

Naïve bayes models didn't perform well. That because the features are numerical and categorical.

- **Explore how preprocessing choices affected model performance.**

The most important prepossessing step in this dataset was encoding the categorical features, as the models can't handle categorical data.

- **Reflect on which model you would recommend for each dataset and why.**

Logistic Regression because it can handle any type of features.